

Structured Computer Architecture
Chapter 2.9: Memory Circuits
First-Order Systems (1-OS)
An Order-Based Approach

John Glossner and Gheorghe M. Ștefan

Based on the textbook

September 14, 2025

Outline

Learning Objectives

By the end of this chapter, you will be able to:

- ▶ Understand the concept of first-order systems (1-OS) and feedback loops
- ▶ Analyze the behavior of latches and flip-flops
- ▶ Design clocked memory circuits and understand timing constraints
- ▶ Explain the operation of RAM and register files
- ▶ Distinguish between synchronous and asynchronous memory systems
- ▶ Understand the trade-offs in memory circuit design

From 0-OS to 1-OS: Adding Memory

- ▶ **0-OS Limitation:** Output depends only on current inputs
 - ▶ No memory of previous states
 - ▶ Cannot store information
 - ▶ Purely reactive behavior
- ▶ **1-OS Innovation:** Introduction of the first feedback loop
 - ▶ Output fed back to input
 - ▶ Creates memory capability
 - ▶ Enables state storage
 - ▶ Partial autonomy achieved
- ▶ **Key Principle:**
 - ▶ Memory function obtained through reverse connection
 - ▶ Circuit maintains state according to command
 - ▶ Foundation for all sequential logic

The Feedback Loop Concept

▶ **Feedback Mechanism:**

- ▶ Output signal returns to influence input
- ▶ Creates dependency on previous state
- ▶ Enables information storage

▶ **State Autonomy:**

- ▶ State can be maintained without continuous input
- ▶ Temporary input can have permanent effect
- ▶ System "remembers" past events

▶ **Applications in Computing:**

- ▶ Register storage in processors
- ▶ Memory cells in RAM
- ▶ State machines in control units
- ▶ Pipeline registers

The Simplest Memory Element

- ▶ **Latch Function:** Store temporary signal transitions
 - ▶ Capture transition from 1 to 0 (reset-only latch)
 - ▶ Capture transition from 0 to 1 (set-only latch)
 - ▶ Maintain state indefinitely
- ▶ **Basic Principle:**
 - ▶ Feedback loop from output to input
 - ▶ Signal propagation through circuit
 - ▶ State becomes self-sustaining
- ▶ **Limitation:**
 - ▶ Simple latches cannot "forget"
 - ▶ Need combination for full functionality
 - ▶ Asymmetric command signals

AND and OR Loop Latches

AND Loop (Reset-Only):

- ▶ Output connected to one AND input
- ▶ Reset' pulse applied to other input
- ▶ Captures transition to 0
- ▶ Output fixed at 0 after reset

Behavior:

- ▶ Normal state: Output = 1
- ▶ Reset' = 0: Forces output to 0
- ▶ Output remains 0 permanently
- ▶ Cannot return to 1

OR Loop (Set-Only):

- ▶ Output connected to one OR input
- ▶ Set pulse applied to other input
- ▶ Captures transition to 1
- ▶ Output fixed at 1 after set

Behavior:

- ▶ Normal state: Output = 0
- ▶ Set = 1: Forces output to 1
- ▶ Output remains 1 permanently
- ▶ Cannot return to 0

Set-Reset (SR) Latch

- ▶ **Combination Solution:** Merge set-only and reset-only latches
 - ▶ Provides both set and reset capability
 - ▶ Can switch between two stable states
 - ▶ Fundamental memory structure
- ▶ **Operation:**
 - ▶ **Set command (S):** Active high, sets output to 1
 - ▶ **Reset command (R'):** Active low, resets output to 0
 - ▶ **Asymmetric commands:** Different activation levels
- ▶ **State Table:**

S	R'	Q	Action
0	1	Q	Hold
1	1	1	Set
0	0	0	Reset
1	0	?	Forbidden

Symmetric SR Latches

NAND SR Latch:

- ▶ Uses De Morgan's law transformation
- ▶ Both commands active low (S' , R')
- ▶ More common in practice
- ▶ Symmetric command structure

Truth Table:

S'	R'	Q
1	1	Hold
0	1	1
1	0	0
0	0	Invalid

NOR SR Latch:

- ▶ Alternative De Morgan transformation
- ▶ Both commands active high (S , R)
- ▶ Symmetric high-active commands
- ▶ Equivalent functionality

Truth Table:

S	R	Q
0	0	Hold
1	0	1
0	1	0
1	1	Invalid

The First Latch Problem

► Problem Statement:

- Same inputs specify **how** and **when** latch switches
- Immediate reactivity to input changes
- Susceptible to noise and glitches
- Mechanical switch bouncing issues

► Consequences:

- Unpredictable state changes
- Noise-induced switching
- Difficult timing control
- Unreliable operation

► Solution Approach:

- Separate "how" from "when"
- Use clock signal for timing control
- Condition inputs with clock
- Achieve synchronous operation

Clocked SR Latch

- ▶ **Design Solution:** Add clock conditioning
 - ▶ S and R inputs specify **how** to switch
 - ▶ Clock (CK) input specifies **when** to switch
 - ▶ Decoupling of timing from function
 - ▶ Synchronous state changes
- ▶ **Implementation:**
 - ▶ Two additional NAND gates
 - ▶ S and R conditioned by clock
 - ▶ Latch responds only when $CK = 1$
 - ▶ Immune to input changes when $CK = 0$
- ▶ **Operation:**
 - ▶ **CK = 0:** Latch holds current state
 - ▶ **CK = 1:** Latch responds to S and R
 - ▶ Eliminates noise sensitivity
 - ▶ Provides controlled timing

D Latch (Data Latch)

- ▶ **Motivation:** Eliminate forbidden state of SR latch
 - ▶ Single data input instead of S and R
 - ▶ No invalid input combinations
 - ▶ Simpler interface
- ▶ **Implementation:**
 - ▶ D input connected to S
 - ▶ $\overline{D}$ connected to R through inverter
 - ▶ Clock enables data transfer
 - ▶ Transparent when clock is high
- ▶ **Behavior:**
 - ▶ **CK = 1:** Q follows D (transparent)
 - ▶ **CK = 0:** Q holds last value
 - ▶ No forbidden states
 - ▶ Simple data storage

From Level to Edge Triggering

- ▶ **Latch Limitation:** Level-sensitive operation
 - ▶ Transparent while clock is active
 - ▶ Multiple changes during clock high
 - ▶ Timing window issues
- ▶ **Flip-Flop Advantage:** Edge-triggered operation
 - ▶ Changes only on clock edge
 - ▶ Single transition per clock cycle
 - ▶ Precise timing control
 - ▶ Better noise immunity
- ▶ **Implementation Methods:**
 - ▶ Master-slave configuration
 - ▶ Edge detection circuits
 - ▶ Pulse-triggered designs

Master-Slave D Flip-Flop

- ▶ **Structure:** Two latches in series
 - ▶ Master latch: Enabled when $\text{CLK} = 1$
 - ▶ Slave latch: Enabled when $\text{CLK} = 0$
 - ▶ Inverted clock between stages
- ▶ **Operation Sequence:**
 1. $\text{CLK} = 1$: Master captures input D
 2. $\text{CLK} = 0$: Slave captures master output
 3. Output changes only on falling edge
 4. Input isolated from output
- ▶ **Advantages:**
 - ▶ Edge-triggered behavior
 - ▶ No transparency issues
 - ▶ Predictable timing
 - ▶ Standard building block

Flip-Flop Types and Applications

▶ **D Flip-Flop:**

- ▶ Data storage and transfer
- ▶ Pipeline registers
- ▶ State machine implementation
- ▶ Most common type

▶ **JK Flip-Flop:**

- ▶ No forbidden states
- ▶ Toggle capability
- ▶ Counter applications
- ▶ More complex control

▶ **T Flip-Flop:**

- ▶ Toggle operation
- ▶ Frequency division
- ▶ Counter circuits
- ▶ Simple interface

Critical Timing Parameters

- ▶ **Setup Time (t_{setup}):**
 - ▶ Minimum time data must be stable before clock edge
 - ▶ Ensures proper data capture
 - ▶ Technology dependent
- ▶ **Hold Time (t_{hold}):**
 - ▶ Minimum time data must remain stable after clock edge
 - ▶ Prevents data corruption
 - ▶ Usually shorter than setup time
- ▶ **Propagation Delay (t_{pd}):**
 - ▶ Time from clock edge to output change
 - ▶ Determines maximum clock frequency
 - ▶ Includes internal delays
- ▶ **Clock-to-Q Delay (t_{cq}):**
 - ▶ Specific propagation delay measurement
 - ▶ Clock edge to output transition
 - ▶ Critical for timing analysis

Timing Violations and Solutions

▶ **Setup Time Violation:**

- ▶ Data changes too close to clock edge
- ▶ Results in metastability
- ▶ Solutions: Slower clock, faster logic

▶ **Hold Time Violation:**

- ▶ Data changes too soon after clock edge
- ▶ Causes incorrect data capture
- ▶ Solutions: Add delay, buffer insertion

▶ **Clock Skew Effects:**

- ▶ Different clock arrival times
- ▶ Can cause timing violations
- ▶ Solutions: Clock tree synthesis, buffers

▶ **Metastability:**

- ▶ Unstable intermediate state
- ▶ Occurs with timing violations
- ▶ Solutions: Synchronizers, proper timing

RAM Fundamentals

- ▶ **Definition:** Random Access Memory
 - ▶ Any location accessible in constant time
 - ▶ Address-based data retrieval
 - ▶ Read and write operations
 - ▶ Parallel extension of 1-OS
- ▶ **Basic Structure:**
 - ▶ 2^p memory locations
 - ▶ p -bit address input
 - ▶ Data input and output
 - ▶ Write enable control
- ▶ **Key Components:**
 - ▶ Address decoder (DEMUX)
 - ▶ Memory cell array
 - ▶ Output multiplexer (MUX)
 - ▶ Control logic

RAM Operation and Timing

► Read Operation:

- Combinational process
- Address selects memory cell
- Data appears at output
- Access time: t_{ACC}

► Write Operation:

- Requires write enable signal
- Address must be stable
- Data must meet timing requirements
- More complex than read

► Critical Timing Parameters:

- t_{ASU} : Address setup time
- t_{DSU} : Data setup time
- t_{DH} : Data hold time
- t_W : Write enable width
- t_{AH} : Address hold time

Multi-bit Word RAM

► **Word-Oriented Memory:**

- Store m -bit words instead of single bits
- Parallel columns for each bit position
- Shared address decoder
- Independent data paths

► **Implementation:**

- One column per bit position
- Each column contains 2^P memory cells
- Common address decoding
- Parallel read/write operations

► **Advantages:**

- Efficient data organization
- Reduced addressing overhead
- Better bandwidth utilization
- Matches processor word size

Synchronous RAM (SRAM)

- ▶ **Motivation:** High-speed operation challenges
 - ▶ GHz frequencies make timing difficult
 - ▶ Asynchronous timing constraints hard to meet
 - ▶ Need better controllability
- ▶ **Synchronous Solution:**
 - ▶ All timing relative to clock edge
 - ▶ Setup and hold times defined for clock
 - ▶ More rigorous timing control
 - ▶ Better system integration
- ▶ **Benefits:**
 - ▶ Simplified timing analysis
 - ▶ Better noise immunity
 - ▶ Easier system design
 - ▶ Predictable behavior

Pipelined Synchronous RAM

- ▶ **Performance Challenge:** Access time too high
 - ▶ Memory access limits clock frequency
 - ▶ Need faster response time
 - ▶ Trade-off between speed and latency
- ▶ **Pipeline Solution:**
 - ▶ Add output register
 - ▶ Fast access time (register delay)
 - ▶ One clock cycle latency
 - ▶ Higher system frequency possible
- ▶ **Design Considerations:**
 - ▶ Latency vs. throughput trade-off
 - ▶ Pipeline depth optimization
 - ▶ Latency hiding techniques
 - ▶ System-level impact

Register File Concept

- ▶ **Definition:** Battery of registers with multiple access ports
 - ▶ Multiple simultaneous read ports
 - ▶ Single write port
 - ▶ Any register accessible each cycle
 - ▶ Critical processor component
- ▶ **Typical Configuration:**
 - ▶ Two read ports (left and right)
 - ▶ One write port
 - ▶ 2^n registers of m bits each
 - ▶ Independent address inputs
- ▶ **Applications:**
 - ▶ CPU general-purpose registers
 - ▶ Pipeline stage buffers
 - ▶ Vector processor registers
 - ▶ DSP accumulator banks

Register File Implementation

▶ **Multi-Port Structure:**

- ▶ Two output multiplexers for reads
- ▶ Address decoders for write selection
- ▶ Write enable gating
- ▶ Simultaneous operation capability

▶ **Interface Signals:**

- ▶ leftAddr: Left read port address
- ▶ rightAddr: Right read port address
- ▶ destAddr: Write destination address
- ▶ writeEnable: Write control signal
- ▶ leftOut, rightOut: Read data outputs
- ▶ in: Write data input

▶ **Design Trade-offs:**

- ▶ Area vs. performance
- ▶ Number of ports vs. complexity
- ▶ Register count vs. access time

Memory Hierarchy Considerations

- ▶ **Speed vs. Capacity Trade-off:**
 - ▶ Faster memory is more expensive
 - ▶ Larger memory is slower
 - ▶ Need hierarchical approach
- ▶ **Memory Types by Speed:**
 - ▶ **Registers:** Fastest, smallest capacity
 - ▶ **Cache:** Fast, medium capacity
 - ▶ **Main Memory:** Medium speed, large capacity
 - ▶ **Storage:** Slowest, largest capacity
- ▶ **Design Principles:**
 - ▶ Locality of reference
 - ▶ Temporal and spatial locality
 - ▶ Caching strategies
 - ▶ Prefetching techniques

Memory Interface Standards

▶ **SRAM Interfaces:**

- ▶ Synchronous operation
- ▶ Clock-based timing
- ▶ Burst mode capability
- ▶ Pipeline-friendly

▶ **DRAM Interfaces:**

- ▶ Row/column addressing
- ▶ Refresh requirements
- ▶ Higher density
- ▶ Complex timing

▶ **Emerging Technologies:**

- ▶ Non-volatile memories
- ▶ 3D memory structures
- ▶ Processing-in-memory
- ▶ Quantum memory concepts

Key Concepts Review

▶ **First-Order Systems (1-OS):**

- ▶ Introduction of feedback loops
- ▶ Memory capability through state storage
- ▶ Foundation for sequential logic

▶ **Memory Elements:**

- ▶ Latches: Level-sensitive storage
- ▶ Flip-flops: Edge-triggered storage
- ▶ Timing constraints and analysis

▶ **Memory Systems:**

- ▶ RAM: Random access capability
- ▶ Synchronous operation benefits
- ▶ Register files for multi-port access

Design Principles and Trade-offs

▶ **Timing Considerations:**

- ▶ Setup and hold time requirements
- ▶ Clock skew and jitter effects
- ▶ Synchronous vs. asynchronous design

▶ **Performance Optimization:**

- ▶ Pipeline techniques for speed
- ▶ Latency vs. throughput trade-offs
- ▶ Memory hierarchy design

▶ **System Integration:**

- ▶ Interface standardization
- ▶ Timing closure methodology
- ▶ Power and area considerations

Connection to Higher-Order Systems

▶ **Building Blocks for 2-OS (Automata):**

- ▶ State storage elements
- ▶ Control signal generation
- ▶ Sequence detection capability

▶ **Foundation for 3-OS (Processors):**

- ▶ Register files for data storage
- ▶ Pipeline registers for instruction flow
- ▶ Memory systems for program/data storage

▶ **System-Level Impact:**

- ▶ Performance bottlenecks
- ▶ Power consumption
- ▶ Reliability and fault tolerance

Questions and Discussion

Questions?

Discussion Topics:

- ▶ How does feedback create memory in digital circuits?
- ▶ What are the advantages of edge-triggered over level-sensitive storage?
- ▶ Why is synchronous design preferred for high-speed systems?
- ▶ How do timing constraints affect system performance?

Next: Chapter 2.10 - Automata: Second-Order Systems (2-OS)